

Cellery: Gradual, Composable Structure for Spreadsheets

Jacob Yim

University of California San Diego
La Jolla, California, USA
jlyim@ucsd.edu

Brian Hempel

University of California San Diego
La Jolla, California, USA
bhempel@ucsd.edu

Sathwika Peechara

University of California San Diego
La Jolla, California, USA
speechara@ucsd.edu

Michael Coblenz

University of California San Diego
La Jolla, California, USA
mcoblenz@ucsd.edu

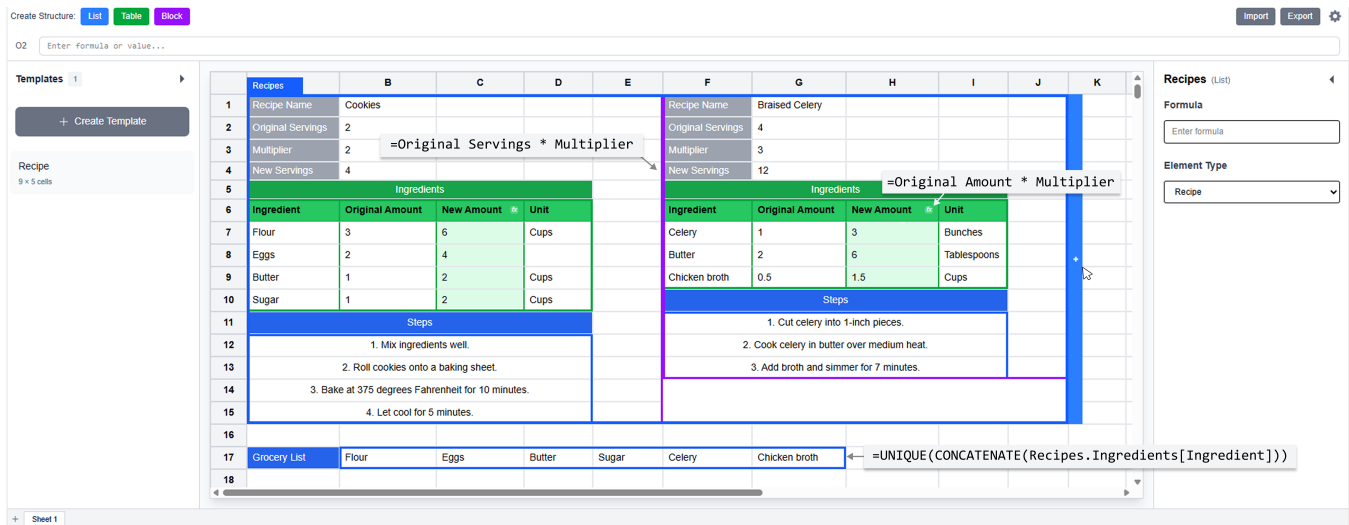


Figure 1: In Cellery, spreadsheets can be structured by combining four structure types: Atoms, Lists, Tables, and Blocks.

Abstract

Structure in spreadsheets is typically restricted to ranges of cells and flat tables. However, replacing the open-ended two-dimensional grid with a more structured alternative risks limiting users' freedom and impeding exploratory workflows. How can we enable expressive structure in spreadsheets while preserving their flexibility? We propose a model that allows users to gradually add structure on top of the traditional grid. To identify common structural patterns, we conducted a content analysis of 30 real-world spreadsheets. Based on our findings, we developed Cellery, a novel spreadsheet system that makes these structures explicit. In Cellery, users gradually define the structure of their spreadsheets using named groupings of cells, composing together four structural primitives: Atoms, Lists, Tables, and Blocks. Once defined, users can directly manipulate structures, dynamically reference them in formulas, and incorporate them in reusable abstractions. We evaluate Cellery with an

expressiveness study reproducing spreadsheets from our sample, demonstrating that it is capable of representing types of structure that traditional spreadsheets cannot. In a preliminary user study of five participants, we find that they successfully use Cellery to design spreadsheets with explicit structure, and we uncover opportunities for further work on the formula language.

CCS Concepts

• Human-centered computing → Graphical user interfaces; • Applied computing → Spreadsheets; • Software and its engineering → Visual languages.

Keywords

Spreadsheets, End-User Programming, Gradual Structure



This work is licensed under a Creative Commons Attribution 4.0 International License. [UIST '26, Detroit, MI, USA](https://creativecommons.org/licenses/by/4.0/)

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2856-3/2026/11

<https://doi.org/10.1145/3830398.3830625>

ACM Reference Format:

Jacob Yim, Sathwika Peechara, Brian Hempel, and Michael Coblenz. 2026. Cellery: Gradual, Composable Structure for Spreadsheets. In *The 39th Annual ACM Symposium on User Interface Software and Technology (UIST '26)*, November 02–05, 2026, Detroit, MI, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3830398.3830625>

1 Introduction

Spreadsheets, used by tens of millions of people [36], are ubiquitous for working with data. The spreadsheet’s open-ended two-dimensional grid of cells enables users’ exploratory workflows and preserves their sense of agency [10]. The freeform grid lets users add data without needing to specify any structure upfront. Through direct manipulation and instant visual feedback, even inexperienced users without deep knowledge of spreadsheet features (e.g., the formula language) can quickly begin working with the system and gently progress to more advanced skills as needed [30].

Many tasks require working with rich structures like nested groups, hierarchies, or heterogeneously-sized collections: all types of data that may be difficult to fit into a flat, two-dimensional form. Unfortunately, spreadsheet users must choose between two unsatisfactory ways of representing these structures. They can impose visual structure with styling, such as bold cell borders; however, these informal representations of structure are invisible to the system, and not usable in contexts like formulas. Alternatively, they can flatten their data, but these transformations may be difficult and complicate analysis because they impact visual layout.

This lack of expressivity has consequences for users. First, since the data model of a spreadsheet is a flat grid, formulas must be written in terms of grid coordinates instead of domain structure. Even with features like named columns in tables, formulas using other types of structure, like hierarchies and nested groupings, may be difficult to write without hard-coded references to cell addresses. Formulas of this kind are fragile: structural change (for example, addition of new data) may cause cell addresses to shift and formulas to break. This can lead to errors and increases the cost of spreadsheet maintenance and reorganization, since formulas must be edited whenever structure changes. Structural *reuse* presents another limitation: when a structural pattern needs to be replicated, it must be copied and edited manually, with the system blind to relationships between instances. Finally, user *interactions* with structure are limited to what the system can represent: beyond flat tables and ranges, users cannot directly select or operate on structure.

As a result, there have been several efforts over the years to introduce more structured approaches to spreadsheets. Prior systems have committed to constrained structural models, such as hierarchies [12, 25], object-oriented classes [16], or relational schemas [5]. While these tools may be especially valuable for users working with specific types of data, they also replace the general-purpose, flexible grid that is so vital to the spreadsheet’s success. Spreadsheet work is frequently exploratory and unplanned [8, 33]: users may begin a task without knowing what the structure of their data will look like, developing their understanding through loosely-structured experimentation. Replacing the open-ended, two-dimensional grid may be too significant a loss of flexibility for many users [10].

Our research question is: *how can spreadsheets support expressive structure without sacrificing their flexibility?* Between the unconstrained grid and prescriptive, predefined structure, is there a middle ground? One answer may lie in the ways users currently structure their spreadsheets. We hypothesize that spreadsheets *already* carry rich structure, but that it is left *implicit* and invisible to the system. We propose a model of *gradual* structure that builds on top of the

flexible two-dimensional grid instead of replacing it, allowing users to incrementally add structure on top of existing data.

To understand the types of implicit structure spreadsheet users currently create, we first conducted a content analysis of 30 real-world spreadsheets from the EUSES corpus [19]. We identified common organizational patterns and formulated design opportunities for improving the expressive power of structure in spreadsheets.

These observations informed our design of *Cellery*, a novel spreadsheet system that allows users to gradually define structure directly on the grid. Rather than a flat grid of values addressed by location, *Cellery*’s data model consists of named structures that reside on the grid. Users compose together four primitive structure types: Atoms, Lists, Tables, and Blocks. Atoms hold single values and are created when users edit a grid location; Lists are one-dimensional, ordered collections of structures; Tables are two-dimensional grids of structures; and Blocks are groupings of structures without ordering or tabular organization. Once defined, these structures provide various capabilities: they can be referenced by name in formulas, directly manipulated to perform operations like expansion, and incorporated into reusable abstractions over structural patterns.

Figure 1 shows a spreadsheet in *Cellery* consisting of a collection of recipes, each with a table of ingredients and list of instructions. A multiplier on each recipe changes the number of servings produced and ingredients needed. *Cellery* supports nested, heterogeneous structure: this spreadsheet consists of a List of Blocks, each containing a Table, a List, and several Atoms. Structures can be given names, which can be placed on the grid as Label Atoms (as shown in Recipe Name, Steps, etc.). Formulas use these names to reference structures: a Table column formula, `=Original Amount * Multiplier`, references the Atom labeled `Multiplier` to compute the count of each ingredient. A construct called a *template* defines a reusable structural pattern: this spreadsheet incorporates a custom Recipe template, so pressing a single “+” button automatically expands the List and creates a new Recipe instance.

Through an expressiveness study using spreadsheets from our corpus, we demonstrate that *Cellery* can explicitly represent forms of structure that remain implicit in traditional spreadsheets. Finally, with a preliminary user study of five participants, we find that participants use *Cellery* to design spreadsheets with rich structure, but may struggle to leverage that structure using formulas.

We focus this work on the design of *Cellery*’s structural model and its interface. *Cellery*’s formula language demonstrates what structure could enable; we leave more focused design and evaluation of the language to future work (Section 8).

This paper contributes:

- (1) A content analysis of 30 real-world spreadsheets, yielding structural patterns that motivate our design.
- (2) A model of composable structure for spreadsheets, featuring four primitives (Atoms, Lists, Tables, and Blocks) that together form expressive structure on the grid.
- (3) A novel spreadsheet system, *Cellery*, implementing our composable structure model paired with structure-aware formulas, interactions, and custom abstractions.
- (4) An expressiveness evaluation reproducing spreadsheets from our content analysis, alongside a preliminary user study of the system (n=5).

2 Related Work

We discuss related work in three categories: approaches for supporting structure within traditional spreadsheet systems; alternatives to the two-dimensional grid; and empirical studies of spreadsheets.

Traditional spreadsheets. Popular spreadsheet systems like Excel and Google Sheets offer several features for adding structure on top of the two-dimensional grid. *Named ranges* allow for cell regions to be referenced in formulas by name rather than by address. *Tables* associate column names with data: for instance, the formula `=AVERAGE(Table1[Column1])` computes the average of `Column1`, regardless of the number of rows. *Dynamic array formulas* return multiple values that “spill” into adjacent cells. *Pivot tables* summarize tabular data into aggregate views.

These features provide helpful structure, but they ultimately still work within the confines of the flat, cell-based model. Most notably, they do not capture the nested, hierarchical structure that Cellery’s compositional model provides. Named ranges carry no structural information about the cells inside beyond their location; additionally, they do not possess explicit operations for expanding or adding data. Tables are most similar in approach to Cellery’s structures; columns can be referenced by name rather than location, and tables can also be transformed with structure-aware operations (e.g. add columns, move column, move table). However, they are still limited to flat layouts, and cannot directly represent nesting or hierarchy. Dynamic arrays are functionally similar to Cellery formulas that return Lists, but they are unaware of surrounding structure: formulas that spill onto existing values cause errors instead of accommodating them. Finally, pivot tables are powerful for summarization, but they are view-only and require source data to be flat: they are a means of viewing data, not structuring it.

One structural extension to spreadsheets proposed by prior work is the Gridlet [21], an Excel function outputting a live copy of a range of cells with value overrides. Gridlets, similarly to Cellery’s templates (Section 6.4), serve as reusable abstractions over spreadsheet regions. The main difference is that Cellery’s templates abstract over composable structures rather than flat ranges of cells, enabling variable-length substructures in templates. Additionally, Cellery supports abstract template definitions with configurable defaults, rather than requiring references to existing instances.

Inferring structure in spreadsheets has been useful for identifying errors in formula logic and references [1–3, 22, 23, 29, 38]. Prior studies show that certain types of structure are common: one study identified 3,290 tables with headers in 3,110 spreadsheets [38]. These works demonstrate that spreadsheets carry valuable latent structure; Cellery extends this idea by allowing users to explicitly define new forms of structure, rather than inferring them.

Alternatives to the two-dimensional grid. Several research systems have addressed the limitations of the flat grid by introducing richer structural models [5, 6, 9, 11, 12, 25–28]. Espalier, also known as Object Spreadsheets [25], replaces the grid with hierarchical collections of typed objects, with columns in the spreadsheet representing attributes and nested groups of columns representing objects. Related Worksheets and its successors SIEUFERD and Ultorg [5, 6] use the spreadsheet as an interface for a relational database, representing one-to-many and many-to-many relationships

via cells holding multiple values, alongside a formula language for constructing SQL-like queries. Gneiss [11, 12] makes externally-structured hierarchical data (e.g., JSON retrieved from web APIs) navigable and manipulable within a spreadsheet interface. The spreadsheet grid itself adopts a hierarchical structure, with cells nested inside grouped rows. The key difference between Cellery and these existing systems is that they all commit to a particular organizational paradigm, whether object-oriented (Espalier), relational (SIEUFERD), or hierarchical (Gneiss). These tools provide deep support for specific types of structure; while this may be useful in specific domains, specialized structure can limit users who do not yet know the organization of their spreadsheet, experiment with different approaches, or work with different types of structure simultaneously. In contrast, structure in Cellery is composable, heterogeneous, and optional: our four primitives can be freely combined to represent diverse organizational patterns, allowing users to define their own structure rather than adopt a paradigm prescribed by the tool. Additionally, some systems, like Espalier, Forms/3 [9], Sumwise [26], and the Lish data model [20], fully replace the two-dimensional grid with a more structured or restricted alternative, which may limit user freedom and introduce premature commitment [7] when structure must be specified up front. Gradual Structuring [28], which helped inspire Cellery’s gradual approach (Section 4.1), begins as a flat grid but still introduces alterations (e.g., hierarchical rows/columns) when structure is added. Cellery, in contrast to these systems, builds structure directly *on top* of the familiar grid rather than replacing it.

Another direction is model-driven spreadsheet development, which involves generating provably correct spreadsheets from specifications [4, 13–18, 31]. For instance, ClassSheets [16] generates spreadsheets from object-oriented, UML-based templates. These systems all involve a separate model specifying spreadsheet structure: users must maintain a schema that is divorced from the main grid, and must express structure using that schema’s specification language. Cellery’s structures, by contrast, live directly on the grid alongside data; there is no separate specification artifact.

Empirical studies of spreadsheets. In spite of much work on spreadsheet errors [32–35], few prior works have used an open-ended approach to study *types of structure* in spreadsheets. A manual analysis [37] identified tabular patterns in spreadsheets from the EUSES and Enron corpora, finding various types of tables including vertical, single-entry, relationship, hierarchical, and replicated tables. These findings are consistent with ours regarding diversity in tabular structures and repeated structural patterns. Liang et al. [24] focused specifically on spreadsheet templates, using an open coding methodology to find that spreadsheet usability is influenced by requirements and structure. They identify two levels of spreadsheet structure, spreadsheet-level and table-level, as well as two types of tables, data tables and insight tables (summaries of the data). Unlike these prior studies, our content analysis examines patterns in the layout of spreadsheets broadly, beyond tabular structures.

3 Content Analysis

To design Cellery’s foundational structures, we conducted a formative content analysis to discover: *What types of implicit organizational structures appear in real-world spreadsheets?*

Table 1: The codebook of structural patterns identified during our content analysis (n = 30 workbook files), sorted by frequency. “# Files” refers to the number of files in our sample containing at least one instance of the corresponding structure.

Category	# Files (%)	Description
Table	29 (97%)	A rectangular region organized into rows and columns, with rows representing records.
Column headers	29 (97%)	A row of cells labeling columns.
Label cells	22 (73%)	A cell whose content names a nearby cell or group (e.g., “Total:”). Does not include table headers.
Row headers	12 (40%)	A column of cells labeling rows.
Header hierarchy	12 (40%)	Headers organized into multiple levels, with grouped subsets of columns or rows.
Merged	8 (27%)	Hierarchy indicated by merging adjacent header cells.
Non-merged	7 (23%)	Hierarchy indicated by styling alone (e.g., indentation).
1D collection	11 (37%)	An ordered sequence of cells arranged linearly, without tabular organization.
Non-tabular group	11 (37%)	A grouping of nearby, semantically related cells whose organization is not tabular.
Nested structure	10 (33%)	A containment relationship in which one structure is spatially and semantically inside of another.
Repeated pattern	10 (33%)	The same structural arrangement appearing multiple times.

Balance Sheet Retail Store, Inc. As at Apr 30, 2001			
Assets		Liabilities	
Cash	\$ 8,050	Loan Payable	\$ 15,000
Accounts Receivable	\$ -	Taxes Payable	\$ 150
Inventory	\$ 13,750	Total Liabilities	\$ 15,150
Promotional Material	\$ 3,000		
Equipment	\$ 27,000	Owners Equity	
Leasehold Renovations	\$ 1,900	Retained Earnings	\$ 8,400
		Owners Investment	\$ 15,000
Total Assets	\$ 53,700	Total Liabilities and Owners Equity	\$ 53,700

Figure 2: A sampled spreadsheet with labels, 1D collections, non-tabular groupings, row headers, and nested structure.

Method. First, we collected 30 spreadsheets by randomly sampling files from the EUSES corpus [19], a dataset of 4,498 real Excel spreadsheets. Because we are interested primarily in understanding what types of structures are present, rather than the distribution of counts of structure types, we chose one spreadsheet workbook (i.e., one .xls file, potentially with multiple sheets) as the unit of analysis for our study. During the coding process, for each file, we indicated whether or not each code was present.

The first two authors began by analyzing an initial sample of 10 spreadsheets and discussing to identify a set of preliminary codes. Using the preliminary codes, the first two authors coded the 10 spreadsheets independently, then compared and discussed until a consensus was reached about the definition of each code. After the pilot sample, the first author annotated the remaining 20 spreadsheets using the agreed-upon codes. We continued to iterate on the codebook throughout, adding new codes as new patterns were observed. After finishing all 30 spreadsheets, the first author then revisited all previously coded spreadsheets to apply changes made to the codebook. Finally, the second author performed a final pass over the codes. Since we coded collaboratively rather than independently, we do not report inter-rater reliability.

Results. Table 1 displays our codebook of structures and the number of files that leveraged each; Figure 2 displays an example from

our corpus. We identified some codes using visual layout: tables, headers, and merged header hierarchies were largely visually unambiguous. However, other codes required interpreting domain meaning: for instance, distinguishing labels from data, or non-tabular groupings from incidentally adjacent data. No new codes emerged starting after coding about 20 spreadsheets, suggesting that our sample size of 30 spreadsheets reached saturation.

Limitations. We observed a small random sample of 30 spreadsheets (out of a corpus of 4,498 spreadsheets), and as such our findings may not generalize to the whole corpus or other unseen spreadsheets. The EUSES corpus was constructed in 2005 and collected largely via Google search; our analysis may be influenced by the search terms used during construction of the corpus and biased towards older Excel spreadsheets. For instance, Excel tables, introduced in 2003 as Lists, were relatively new at the time of the corpus’s construction and may have led to more standardized table layouts in future years. Our derived codes are largely based on layout, rather than other types of structure (e.g., formula references); we do not claim to identify all forms of implicit structure.

3.1 Opportunities: Representational Gaps

Based on our findings, we identify types of implicit structure that traditional spreadsheet systems cannot represent explicitly. These are *representational gaps*: organizational patterns that users incorporate but that the flat, grid-based model has no means to encode. Each gap presents a design opportunity for improving the structural expressiveness of new systems. We present five such gaps:

Labels on the grid (73% of files). Labels were present in majority of the spreadsheets in our sample. Cells with semantic information may be useful for helping readers understand a spreadsheet. However, in traditional spreadsheets, labels and data are treated identically: both are cells containing values. As a result, the labeling relationship cannot be leveraged by the system, e.g., in formulas.

Hierarchical table headers (40% of files). Traditional spreadsheets support explicit tables with single-level column and row headers, but not header hierarchies. We find that spreadsheets in our sample often incorporate hierarchies indicated by layout and styling: 27% of spreadsheets incorporate hierarchical tables with column or row groups indicated by merged headers, while 23% of spreadsheets have tables with hierarchy indicated by styling

choices, like indentation or bolding. In both cases, hierarchy is not a formal construct, and is instead communicated visually.

Non-tabular groupings of cells (37% of files). Spreadsheets have limited capacity for representing groupings of cells that do not conform to a row-column pattern. For instance, we observed several form-like spreadsheets consisting of structured regions containing heterogeneous sub-structures, such as the spreadsheet displayed in Figure 2. Named ranges capture a rectangular region but provide no information about its internal organization; the flat grid model cannot represent or navigate unordered, heterogeneous containers.

Nested structure (33% of files). Traditional spreadsheets provide no explicit way of representing *nested* structures: structured groups of cells inside of each other. For instance, a flat Excel table cannot represent a table where the row-column intersections hold lists of cells, a pattern that we observed twice in our sample.

Repeated structural patterns (33% of files). When the same structural arrangement appears multiple times, for instance, a series of tables with the same column headers but different data, the relationship between instances is invisible to the system. We observed a variety of patterns in our sample, including structure repeated across tables and the aforementioned non-tabular groupings.

4 System Design Principles

Following our content analysis, we guided the development of our system, Cellery, with two high-level design principles:

DP1: Structure should be **gradually defined**.

DP2: Structure should be **composable**.

These principles work together towards our goal of enabling structural expressiveness without sacrificing flexibility: graduality preserves the flexibility of the two-dimensional grid, while composability enables expressive user-defined structure. We define, motivate, and expand on these goals in the following sections.

4.1 Gradual Structure

To preserve the freedom and flexibility of the two-dimensional grid of cells, we aim to build structure on top of, rather than replace, the familiar grid. We leverage the concept of *gradual structuring* proposed by Miller and Hermans [27, 28], whereby users incrementally add structure to a spreadsheet as needed. To focus our design, we highlight four specific aspects of gradual structure:

Optionality. Structure should be optional, giving users freedom to experiment without committing to a specific model.

Explicit, not inferred. To preserve user agency, structure should not be inferred or introduced without explicit user intent [10]. The system should not prescribe a specific structure for a given use case, but should allow the user to pick from different possibilities.

Top-down and bottom-up. To maximize flexibility and prevent *premature commitment* [7], structure should be specifiable in both a top-down and bottom-up manner: users should be able to introduce structure both before and after adding data.

Preserve layout and organization. Finally, wherever possible, structure should be applicable to data as it already exists on the grid, without having to reorganize it to fit the system. We want to leverage the organizational patterns that people already use, rather than necessitate restructuring to serve a particular model.

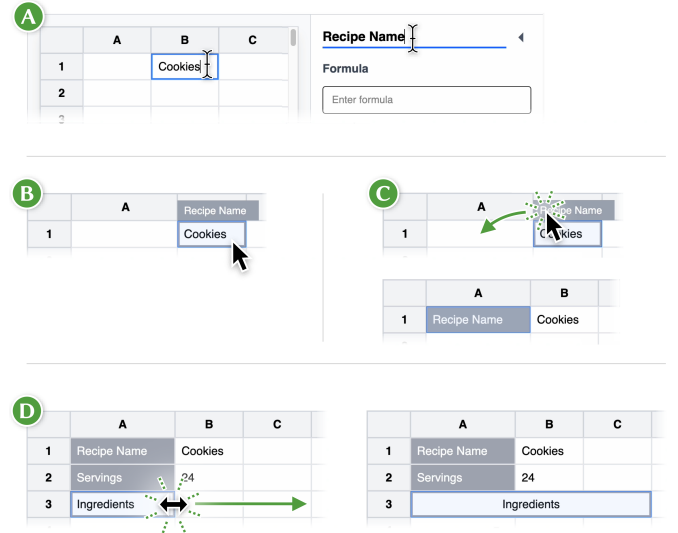


Figure 3: Sam begins building her recipe book by **A** creating and labeling **Atoms** and **D** resizing structures.

4.2 Composable Structure

Our content analysis revealed five opportunities to improve the structural expressiveness of spreadsheet systems. All five of these opportunities synergize with a *model of composable structures*: labels allow for navigation of composed structures; hierarchical headers represent compositions of sub-tables; non-tabular groupings introduce containment-based, potentially heterogeneous composition; nesting is composition by definition; and repeated patterns can be represented as abstractions over compositions of structures.

5 Example Use Case

Guided by these design principles, we developed Cellery, a spreadsheet system with gradual and composable structure. To introduce Cellery, we describe a fictional scenario leading to the spreadsheet in Figure 1. Sam is preparing to cook dinner and wants to build a recipe book to keep track of ingredients to buy and cooking instructions to follow. To start, she records a recipe for cookies.

Atoms and Labels. To title the recipe (Figure 3 A), she double-clicks grid position B1 and types Cookies to create an *Atom*, the basic structure for a single value on the grid (analogous to cells in other spreadsheets). The *structure panel* on the right sidebar lets her name the Atom Recipe Name. By default, structure labels appear as a tab B when a structure is selected or when hovering over the structure. To instead persist the label on the grid, Sam C drags the tab to A1 to reify it as a Label Atom. Labels can also be created in the reverse order: Sam writes Servings and 24 in adjacent Atoms, right-clicks on Servings, selects “Use as structure label” from its context menu (not shown), then clicks the 24 Atom to label it. The link between a Label Atom and the named structure is persistent: changing the text in the Label Atom’s grid location renames the referenced structure, and changing the structure’s name in the structure panel changes the value of the linked Label Atom.

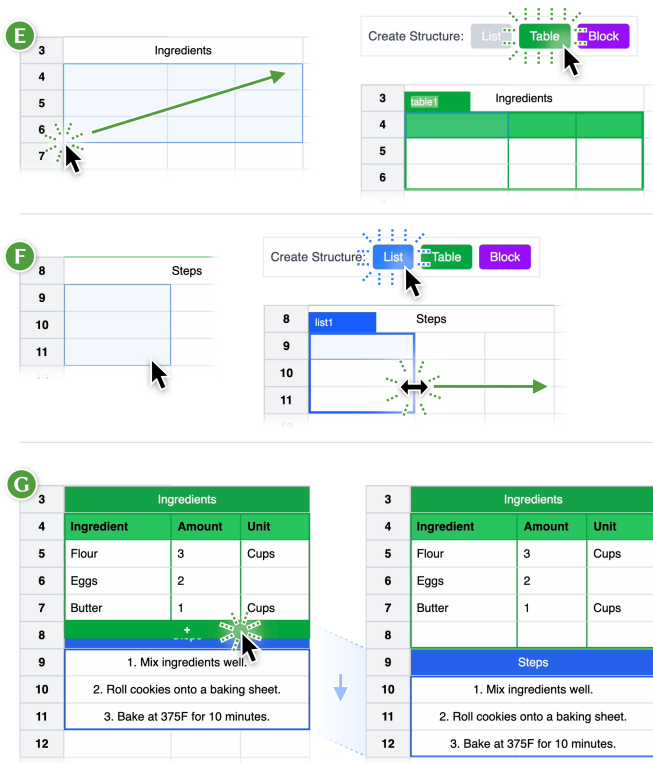


Figure 4: Sam creates structure with **E** a Table and **F** a List. **G** Expanding a structure repositions other structures.

Resizing. When selected, structures can be resized by dragging their edges. For Atoms, resizing merges cells, allowing Sam to **D** set up a title for the forthcoming ingredients table.

Tables and Lists. Because Cellery is gradual, structures can be created first and filled in, or can be applied to existing data. Sam chooses to draw the bounds of her ingredients table ahead-of-time and clicks the “Table” button (Figure 4 **E**), which creates a new Table with a header row. She fills in the Table’s column names and data, and sets the Ingredients Atom to label the Table (not shown).

She **F** creates her ingredients list similarly: the “List” button initializes a range of cells as a new vertical List, which she resizes to be three columns wide, and fills in with the recipe instructions.

Visibility invites interaction with structures and may help users understand the organization of the spreadsheet. Thus, Cellery always displays structures on the grid—the colored boxes are persistent. This contrasts with named ranges in Excel and Sheets, which must be viewed in a menu or a formula that references the range.

When Sam hovers the cursor over the end of a Table or List, a button appears to add another row, column, or item to the structure. Sam **G** uses this button to make a row for a new ingredient. Cellery automatically maintains the layout of her spreadsheet by moving the List of instructions downward to avoid overlap. In general, Cellery follows two layout rules: (1) Overlap is disallowed, so expanding structures push other structures in the direction of expansion, as in this case. This push is recursive: if pushed structures

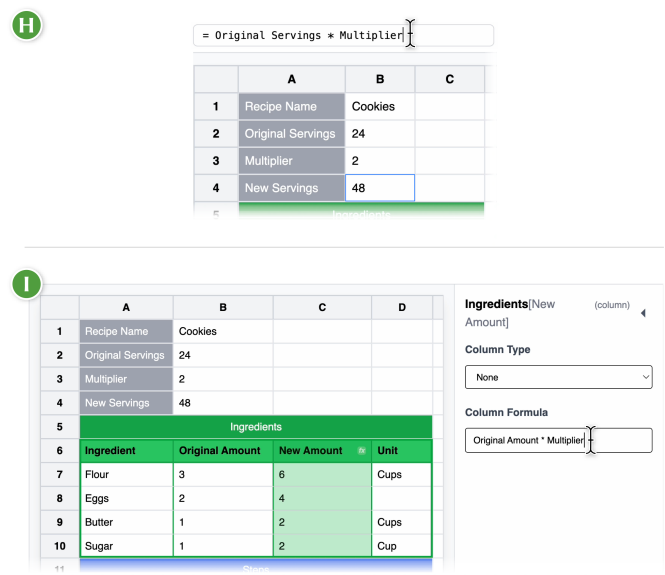


Figure 5: Sam makes her recipe computational using structural formulas in **H** an Atom and **I** a Table column.

are moved into other structures, those structures are also pushed. (2) Parent structures must fully contain their children, so *parents expand with children* if a child expands beyond the parent’s bounds.

Structural Formulas. Sam realizes that she needs to make more servings than her recipe provides. Labeled structures enable Cellery to support *structural formulas* that reference data by name instead of by grid location. After adding an Atom labeled Multiplier, Sam uses the formula =Original Servings * Multiplier to compute a New Servings Atom, as shown in Figure 5 **H**. Columns may be computed by formula as well. Sam adds a column using the context menu (not shown) and **I** uses the structure panel to set a formula for the whole column: Original Amount * Multiplier.

Blocks and Templates. Sam decides to add additional recipes to her spreadsheet. The Block structure in Cellery gathers all the structures within a rectangular region into a single entity. Sam selects a range encompassing the recipe’s cells and presses the “Block” button (Figure 6 **J**). This makes all the contained structures children of the new Block. To faithfully communicate structure to users, Cellery follows the principle that *spatial containment indicates ownership*. Moving a child into or out of a parent will reparent it; this applies to moving structures into and out of Lists and Tables as well. Parenting affects referencing: after Sam names the Block recipe1, her formulas may reference the structures inside: e.g., =recipe1.Recipe Name.

To reuse Blocks, Cellery lets users turn a Block into template, for instance, to represent a blank recipe. Templates provide both a default object for new items when expanding a collection (e.g., a List) as well as a way to bulk edit all instances. The right-click context menu on the recipe1 Block lets Sam initialize a new template, which can be edited by selecting it from the *template panel* in the left sidebar (visible in Figure 1) to open the *template editor* **K**.

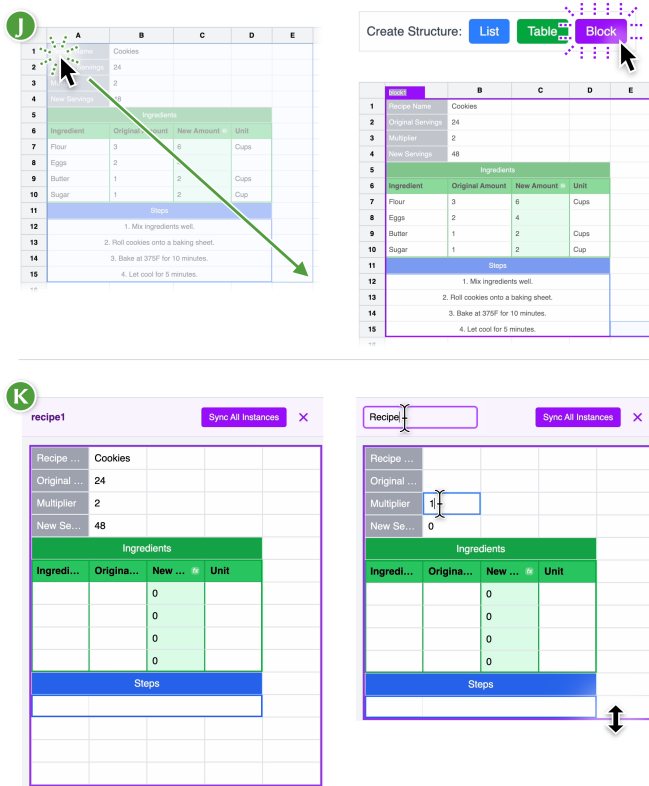


Figure 6: J A rectangular **Block** serves to group heterogeneous structures into a named object which may be abstracted into a template, editable with K the template editor.

Inside the template definition, the Ingredients Table and Steps List have been cleared automatically. She clears the other fields, names the template Recipe, and shrinks the template bounds; instances will later expand to encompass their child structures.

Now, to create a list with multiple recipes, back in the spreadsheet Sam makes a new List, names it Recipes, and (not shown) drags the existing recipe1 Block into the first cell of the List: this configures the List so its elements follow the Block's template, namely Recipe. Now, when Sam hovers over the end of the List and presses the "+" button, a new Recipe instance is automatically created. Sam can now add more recipes without re-specifying their structure, and their content is individually editable. She fills in a recipe for Braised Celery (shown in Figure 1). At this point, the structures are quite deep: e.g., a recipes List containing a Recipe Block containing a Steps List containing an Atom. For focus in the structure panel, when multiple structures occupy the same location, clicks cycle through them in descending order of size.

Computing Over Repetitive Hierarchical Structure. Finally, Sam wants to create a shopping list of all the ingredients. In Cellery, formulas may resolve to whole structures—not just single values—and Lists allow broadcast access to the members' properties. For example, writing the formula `=Recipes.Recipe Name` in a new grid cell will make a computed List, with the recipe names spilling to adjacent grid cells. In a new grid cell below her recipes, Sam writes

a formula leveraging her nested structures:

```
=UNIQUE(CONCATENATE(Recipes.Ingredients[Ingredient]))
```

This formula gets the Ingredients Table of each Block in Recipes, accesses the Ingredient column of each, concatenates them into a single List, and then deduplicates the List. The result is shown at the bottom of Figure 1. Since formulas recompute when referenced structures change, adding recipes or ingredients automatically updates Sam's grocery list. Sam is ready for her trip to the store!

6 System Description

Table 2 displays Cellery's core structural features. In the following sections, we detail the design of our system.

6.1 Structure Model

Cellery's design centers on a hierarchy of named structures representing the underlying organization and content of the spreadsheet. Following DP2, this hierarchy is built by composing four structure types. Each sheet in Cellery is a collection of optionally-named structures, each of which may contain other structures, forming a tree. Some structures are *containers* that store other structures (internal tree nodes); others are *Atoms* that hold values (leaf nodes). Following DP1, this design supports *partial* structural commitment: users can choose to organize some values on the grid into containers while leaving others as loose cells. Cellery tracks the current grid location of each structure, updating that location as needed to account for changes to adjacent structures.

Cellery rests on the principle that *everything on the grid is a structure*, including cells. While traditional spreadsheet systems center the grid by treating features like named ranges and tables as ranges of cells, Cellery's structure-first model reframes cells themselves as structures. All structures have shared properties: they are rectangular, have a position and dimensions, an optional name, and an optional formula. Structures also share many capabilities: they can be interacted with in similar ways, can be referenced by name in formulas and assigned a formula (Section 6.3), and can be incorporated in templates (Section 6.4). This ensures that structures behave consistently: our intent is to allow users to transfer concepts across structures of different types.

6.2 Structure Primitives

In this section, we describe Cellery's four base structure types: the primitives that compose together in our structure model.

Atoms. An Atom is a structure holding a single value, such as a number or a string. From the user's perspective, they function similarly to cells in other spreadsheet systems: users can freely type inside squares on the grid to create them. Atoms may contain either raw values, or they can be assigned a formula. Atoms are the only structures that can contain raw values, so collections storing data must be created by composing Atoms together with other structure types. Atoms can also serve as structure labels.

Lists. Lists are one-dimensional, ordered collections of other structures. They can be either horizontal or vertical, and can be rotated using the context menu. In the spirit of supporting top-down and bottom-up structure, Lists (and other types of containers) can be created on top of existing Atoms, or Atoms can be added

Table 2: Summary of Cellery’s structure types, grounded with codes from our content analysis.

Structure	Description	Code
Atom	A single value on the grid.	N/A
Label	Atom naming another structure.	Label cells
List	1D, ordered collection of structures.	1D collection
Table	2D, tabular collection of structures.	Table
Hierarchical	Table with hierarchical headers.	Header hierarchy
Block	Unordered grouping of structures.	Non-tabular grp.
Template	Reusable Block with default values.	Repeated pattern

by editing grid locations within. By default, Lists contain Atoms; however, to support composability, the *type* of a List can be changed using the structure panel. Lists can contain any structure type (e.g., a List of Tables). However, direct children of Lists must be homogeneously typed: creating a List with differently-typed elements (e.g., a List containing an Atom and a Table) is disallowed. This prevents ambiguity in formula semantics: for instance, `SUM(list1)` produces a single value when `list1` contains Atoms, but the sum of a List containing both Atoms and Blocks would be ill-typed. However, due to Cellery’s lack of a more expressive type system, this restriction does not apply across descendants: a List containing a List of Atoms and a List of Tables, for instance, is allowed.

Tables. Tables are two-dimensional, ordered, optionally axis-labeled collections containing other structures. In our content analysis, we observed tables with a variety of header configurations. As a result, Cellery’s Tables can be labeled with both column and row headers. Like Excel, headers take up grid space, and their values serve as column/row labels. To prevent ambiguity when accessing data, each row-column intersection may contain only one structure.

Like Lists, Tables contain Atoms by default. To further support nested structure, Tables are typed at the column level: a structure type can be assigned to a selected Table column. Table rows and columns can be assigned formulas in a similar way.

Hierarchical tables appeared in 40% of spreadsheets during our content analysis. For example, header columns “2025” and “2026” might each have sub-headers “Q1”–“Q4,” representing four quarters per year. To represent this structure in Cellery, hierarchical Tables can be created using *multi-level headers*. “Levels” can be added to or removed from headers by dragging the edge of the header area. In our content analysis, 66% of spreadsheets incorporating hierarchical tables did so using merged headers: Cellery allows users to merge upper-level header cells by dragging their edges to create *column and row groups*, which can be accessed in formulas.

Blocks. 37% of spreadsheets in our content analysis contained non-tabular groupings of cells. Cellery supports these structures with Blocks, which are unordered rectangular regions on the grid that may contain other structures. Unlike other structure types, Blocks may contain heterogeneously typed children, e.g., an Atom, a List, and a Table. A Block places almost no restrictions on what can be placed inside and functions simply as a grouping of structures on the grid. Blocks are analogous to *records* in functional programming because they consist of named fields. Finally, Blocks can be created from or converted to a template (Section 6.4).

6.3 Structure-Aware Formula Language

At a high level, Cellery’s formula language is similar to that of other spreadsheets, extended with features for referencing and extracting data from our structures. To remain accessible to traditional spreadsheet users, Cellery adopts an Excel-like syntax. However, because structures can change locations, formulas normally reference them by *name*. For example, the formula `=SUM(list1)` references the elements of `list1`. Nevertheless, to maintain compatibility with traditional spreadsheet language features, cell values can also be referenced using geometric addresses: e.g., `A1` or `B1:B3`.

To support references to data inside structures, Cellery uses syntax and semantics inspired by programming languages like Python. Elements of Lists can be accessed by index, e.g., `list1[0]`. Tables can be accessed by column name, e.g. `Ingredients[Ingredient]`. Tables can also be accessed by row index, e.g. `Ingredients[0]`, or by row label, if provided. Table column and row references return results in the form of a List; one limitation of this approach is that rows produced in this way cannot be accessed by column name afterwards, as there is no dedicated structure type for individual Table records. Lastly, structures inside of Blocks can be accessed using dot notation, similarly to object fields in Python: e.g., `block1.struct1`.

Cellery’s formula language supports commonly-used functions (SUM, AVERAGE, COUNT, etc.). Functions may operate over Atoms, Lists, and Tables. Some functions are polymorphic: e.g., the SUM function produces a scalar Atom value when called on a List, but produces a List when called on a Table. Aggregator functions like SUM take an optional axis parameter (DOWN or ACROSS) when called on Tables; when omitted, the axis is inferred based on the direction of the target List. A vertical List automatically aggregates across the rows of a Table, and a horizontal List aggregates down the columns.

Formulas inside containers can leverage their container for scope. Formulas inside a Block can reference other structures in the Block without having to specify the container; formulas assigned to Table columns can reference other columns in the same Table. This also allows formulas to be duplicated across template instances, since each is evaluated in its own context (Section 6.4).

Formulas are capable of returning entire structures as output. Using the structure panel, users can select a structure (e.g., a List) and enter a formula that determines its values. Formulas can also be assigned to Table columns, which behave identically to Lists in the formula language. Additionally, writing a formula returning a structure directly on the grid creates a structure of the formula’s output type. For instance, `=block1` outputs a live copy of the Block onto the grid, including all structures nested inside. Formulas referencing structures recompute automatically when those structures change. However, structures *computed* by formulas are read-only: if a user attempts to edit such a structure, Cellery emits a warning and severs the link between formula inputs and output.

References to row and column groups also evaluate to entire Table structures, enabling aggregate computations. For example, the formula `=AVERAGE(2025)` inside of a Table where 2025 is a group of four columns computes the average across all four columns. A formula with a column group as output, such as `=table1[colGroup]`, returns a whole Table with all of the columns in the group.

Cellery has a *broadcasting* feature for aggregate operations across Lists of Blocks. Inspired by functional programming, users can

map operations across elements of Lists. However, doing so in a traditional functional language might require writing a lambda function, which may not be accessible to spreadsheet users without programming experience. In Cellery, dot notation on a List of Blocks automatically distributes over its elements: `=list1.struct`, where `list1` contains Blocks, returns a List with `struct` from each Block.

6.4 Template System

33% of the spreadsheets in our content analysis incorporated repeated structural patterns. To capture these in Cellery, we implemented *templates*: customizable abstractions over structures that enable pattern reuse. Templates are managed and created using the *template panel* on the left side of the screen (visible in Figure 1). Creating a new template opens up the template editor (shown in Figure 4 [6]), a separate, resizable grid with the same capabilities as the main interface. The template editor contains a template definition: a Block whose structures serve as the pattern to be reused. Values inside a template definition serve as defaults, while empty structures act as placeholders for instance-specific data. Formulas and Labels inside a template are preserved in instances, enabling formula reuse. Once defined, a template can be instantiated by dragging it from the template panel onto the grid, producing a Block that copies all structures from the definition. Instances can be edited individually; changes to the template definition do not propagate automatically to preserve edits users have made to individual instances. A “Sync All Instances” button in the template editor applies updates to all instances, which may overwrite existing data.

To support bottom-up structuring (avoiding premature commitment [7]), a Block can also be converted into a template via the context menu. During conversion, contained Lists and Table rows are cleared, since these structures may hold variable-length data that differs across instances; Atoms retain their values as defaults. Templates can serve as element types for Lists and Table columns: selecting a template from the element type dropdown configures the container so that each element follows the template’s structure. The same can also be accomplished by dragging a template from the panel or an existing template instance into an empty List. Adding new elements to such a List automatically produces new template instances: in the example, pressing the “+” button produces a new, fully structured recipe. This integrates naturally with the formula language’s broadcasting feature, which extracts data across template instances in a structured way.

6.5 Implementation Details

We implemented Cellery as a web application in TypeScript with React. Cellery can import and export spreadsheets as JSON. Excel files can also be imported as unstructured spreadsheets of Atoms.

7 Evaluation

We evaluate Cellery using an expressiveness study and a preliminary usability study.

7.1 Expressiveness Study

We reproduced our original sample of 30 spreadsheets in Cellery to answer the research question: *To what extent does Cellery address the representational gaps identified during our content analysis?*

Figure 7: A spreadsheet, originally in Excel, reproduced in Cellery during our expressiveness study. The spreadsheet has three sheets with repeated structure, which in Cellery are linked together via a template. Of our five representational gaps, this spreadsheet expresses four: nested structure, labels, non-tabular groupings, and repeated patterns.

Methods. For each spreadsheet, the first author examined the original file and (1) imported it into Cellery using Cellery’s Excel import feature; (2) constructed the spreadsheet’s organizational structure in Cellery, creating structures corresponding to those observed during the content analysis. We prioritized structural fidelity over exact appearance, focusing on capturing the underlying organization of each spreadsheet in Cellery’s model. Visual layout was preserved wherever possible; we noted cases needing alteration.

Because the representational gaps identified in Section 3.1 represent the structural limitations that Cellery aims to address, we focus our evaluation around these five organizational features. For each reproduction, we recorded whether each instance of each gap type was successfully expressible in Cellery. To document Cellery’s limitations, we noted any adjustments needed to express each structure in the system, alongside any types of structure that Cellery’s model was not capable of representing.

Results. Every instance of each of the five representational gap types from our content analysis was expressible in Cellery, though some layout adjustments were required, which we now describe. Figure 7 displays an example of a reproduced spreadsheet that expresses four of the five gap types.

Adjustments: Twelve spreadsheets required minor adjustments. Since row and column hierarchies must be specified in Cellery using merged, multi-level headers, any tables whose hierarchy was indicated by styling rather than merging (7 spreadsheets) required adjustment. This typically involved adding a header level to the table, relocating the subordinate rows/columns to a different header level, and merging to produce row/column groups. Tables with rows or columns wider or taller than a single cell but containing

Table 3: Study participant demographics.

Participant #	Spreadsheet Exp.	Programming Exp.	Occupation	Gender	Age	Education
1	Intermediate	Novice	Law student	M	25–34	Bachelor’s
2	Intermediate	Novice	Sustainability	F	18–24	Bachelor’s
3	Intermediate	Expert	Game developer	M	18–24	Bachelor’s
4	Intermediate	Novice	Film	M	25–34	Bachelor’s
5	Beginner	Advanced	Student (CS & Finance)	M	18–24	High school

single values (4 spreadsheets) required compaction, since manually resizing Table rows/columns is not implemented in Cellery. Additionally, one spreadsheet used cell background color to indicate boolean values (true/false encoded by gray/white color); lacking styling features, we converted these to use TRUE/FALSE.

Non-representable structures: Three spreadsheets contained structural features that were not representable in Cellery. One spreadsheet contained row headers on the *right* side of the table, while Cellery only supports headers on the left-hand side. Another spreadsheet included a flowchart constructed from bold cell borders, and the third contained a diagram of a GUI layout. In those documents, the spreadsheet is used as a canvas for drawing rather than a container for grid-structured data; this falls outside our scope.

Limitations. We built Cellery to address the gaps identified during our content analysis: their representability follows from the system’s design, and so the adjustments and non-representable structures characterizing the limits of our model should be viewed as the substantive findings of this expressiveness study. We conducted the study on the same sample on which we based our system: our results may not generalize to unseen spreadsheets. Additionally, due to Cellery’s limited formula language, our study does not evaluate whether *formulas* from the sample are reproducible in Cellery, only types of structure. A deeper analysis with formulas could evaluate our language features and reveal other limitations.

7.2 Preliminary User Study

We conducted a think-aloud usability study asking: *In tasks involving spreadsheet authoring and modification, what kinds of structure do users employ with Cellery, and what difficulties do they encounter when applying structural concepts?*

Participants. We recruited a convenience sample of five participants (four men, one woman), all of whom had some spreadsheet experience. Table 3 displays participant demographics. Participants were compensated with a \$30 gift card for 90 minutes of their time.

Protocol. Participants watched a 16-minute video tutorial introducing Cellery. They then completed three spreadsheet tasks. We asked participants to think aloud and allowed them to ask the researcher questions about the system. We captured recordings of the screen and took notes about each participant’s approach.

Tasks. To elicit gradual structure from participants, we separated tasks into parts that incrementally added complexity to the spreadsheet. The first task involved creating a spreadsheet recording the topic, names of attendees, and attendance count at a recurring meeting, then computing the average number of attendees across all meetings and a list of topics covered. The second task, adapted from

a spreadsheet task used by Liang et al. [24], involved computing grades for a class of students using a weighted average of assignment scores, and then extending the spreadsheet to compute the average grade of multiple classes. The third task involved producing a spreadsheet similar to our example in Figure 1 and Section 5, with a series of recipes containing heterogeneously-structured data and computed ingredient counts based on a scale factor. Full task descriptions can be found in the appendix.

Results. We synthesized observations across our session notes to identify strategies employed by users and difficulties they faced. Table 4 displays a summary of participants’ approaches to the tasks, categorized by types of structure and formulas used. Participants generally used Cellery’s unique features during the tasks, and were able to gradually structure their spreadsheets with various compositions of Cellery’s primitives. Four of the participants used a nested Table of Lists to complete Task 1. All four designed table column formulas that automatically adjusted to the length of the nested lists. The remaining participant used a flat table, but expressed that there was likely a “more efficient way” to perform the task. On Task 2, two participants again used a Table of Lists; three created hierarchical tables. Three participants wrote formulas referencing hierarchical columns or nested Lists in Tables, while two did not. Notably, one participant created hierarchical tables but used Excel-style geometric references to write formulas instead of using the hierarchy, perhaps attributable to discomfort with the formula language; another participant used primarily regular, non-hierarchical table column references. Additionally, to create repeated tabular structures during Task 2, three participants used templates. However, to duplicate a formula, two participants created instances first, then manually changed each instance to add the formula (instead of editing the template and then instantiating it); some participants may underestimate the capabilities of templates. Four participants did not complete Task 3 due to the limited duration of the study. The one participant who did complete the task used repeated templates to do so, but struggled to access data from within nested Blocks, suggesting that the scoping behavior of Blocks may introduce difficulties for users.

Limitations. Some participants encountered bugs, e.g., with formula recomputation; difficulties with these are excluded from our analysis. The short duration of the study is another limitation, as 4/5 participants did not have time to complete Task 3. Due to the think-aloud and open-ended nature of the study, we do not evaluate task completion timing or correctness. Our study was conducted with a relatively small sample of five participants and without comparison to traditional spreadsheets; further investigation is needed to more definitively assess Cellery’s usability and utility.

Table 4: Summary of participants’ approaches to the user study tasks, categorized by the structures and formulas used. Unique features of Cellery are indicated in bold. Note that some traditional spreadsheets (e.g., Excel) support tables and formulas referencing table columns, so these features are not indicated as unique to Cellery.

P#	Task 1		Task 2		Task 3	
	Structures	Formulas	Structures	Formulas	Structures	Formulas
1	Flat Table of Atoms	References to Table columns; geometric references	Blocks containing hierarchical Tables	References to hierarchical Table columns; geometric references	Did not complete	
2	Table containing column of Lists	References to Lists in Tables	Templated Blocks , containing Tables , containing columns of Lists	References to Lists in Tables ; references to Table columns; geometric references	Did not complete	
3	Table containing column of Lists	References to Lists in Tables	Templated Blocks , containing Tables , containing columns of Lists ; Label Atoms	References to Lists in Tables ; references to Atoms in Blocks ; references to Table columns	List of templated Blocks containing Blocks , Tables , Lists , and Atoms	References to Table columns in Blocks ; references to Atoms in Blocks
4	Table containing column of Lists	References to Lists in Tables	Templated Blocks containing hierarchical Tables	References to Table columns; geometric references	Did not complete	
5	Table containing column of Lists	References to Lists in Tables	Blocks containing hierarchical Tables	References to Table columns; geometric references	Did not complete	

8 Discussion and Future Work

Following our content analysis, system design, and evaluation, we draw takeaways about the limits and implications of our system.

Future work on the formula language. From our two-part evaluation, we find that Cellery’s composable model is able to represent types of structure that traditional spreadsheets leave implicit, and moreover that users are capable of incorporating these structures during spreadsheet tasks. However, for the tasks that we gave our participants, we also found that the primary opportunities for improvement lie not in *expressing* structure, but in *leveraging* it. Most participants were able to define rich compositions of structures in Cellery; while the majority of participants for each task were also able to write structural formulas, some participants struggled and fell back on Excel-style geometric references. Similarly, templates had mixed adoption. This points to a distinction between *structural expressiveness* and *structural formula literacy*: Cellery succeeds in its primary goal of making structure more expressive, but users may need more support for writing formulas and managing abstraction in order to effectively utilize richer structure. Future work could investigate how to scaffold the formula authoring experience, especially for spreadsheet users who lack experience programming with structured data. For instance, there may be opportunities for users to leverage structure *without* writing textual formulas: e.g., using direct-manipulation operations.

Our content analysis also did not consider the structural role of formulas, nor did our expressiveness study reproduce formulas; further design and evaluation of the formula language is an important opportunity for future research. For instance, the formula language could be improved with a more expressive type system over references to Cellery’s structures. A formula-focused corpus analysis and user studies investigating the human factors of the language could better evaluate whether formulas in Cellery can leverage structure effectively.

Future work on the structural model. The Cellery model is not the only possible model for gradually structuring spreadsheets. Future work could explore alternative data models: for instance, Tables could be represented as Lists of Records. The experience of structuring spreadsheets could also be further explored: structures in Cellery must be manually created, but future work could investigate how to automatically infer our structures from spreadsheets.

Implications of gradual and composable structure. The problem of users working with implicit structure is not unique to spreadsheets. Other substrates, such as document editors and canvas-based design tools, could support richer computation over and interaction with user-defined structure if it were made explicit.

9 Conclusion

Cellery shows how the implicit structure of spreadsheets can be made explicit using a composable model of structure. Guided by a content analysis of 30 spreadsheets, as well as our principles of composability and graduality, we designed a model consisting of four primitives—Atoms, Lists, Tables, and Blocks—and instantiated it in a system, Cellery, that leverages structure in formulas, interactions, and reusable templates. We evaluated our system by reproducing our sample of 30 spreadsheets, as well as conducting a preliminary user study. By reimagining the spreadsheet model without replacing the grid, our work shows that expressive structure in spreadsheets can be achieved without sacrificing flexibility.

Acknowledgments

We thank the anonymous reviewers for their thoughtful feedback, as well as our user study participants for their time and insights. We also thank Joshua Horowitz, Devamardeep Hayatpur, the SALAD Lab, and the Pacific Programming Interfaces Confab for many valuable discussions that helped shape this work.

References

- [1] R. Abraham and M. Erwig. 2004. Header and Unit Inference for Spreadsheets Through Spatial Analyses. In *2004 IEEE Symposium on Visual Languages - Human Centric Computing*. 165–172. doi:10.1109/VLHCC.2004.29
- [2] Robin Abraham and Martin Erwig. 2006. Inferring templates from spreadsheets. In *Proceedings of the 28th international conference on Software engineering (ICSE '06)*. Association for Computing Machinery, New York, NY, USA, 182–191. doi:10.1145/1134285.1134312
- [3] Robin Abraham and Martin Erwig. 2007. UCheck: A spreadsheet type checker for end users. *Journal of Visual Languages & Computing* 18, 1 (Feb. 2007), 71–95. doi:10.1016/j.jvlc.2006.06.001
- [4] R. Abraham, M. Erwig, S. Kollmansberger, and E. Seifert. 2005. Visual specifications of correct spreadsheets. In *2005 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC'05)*. 189–196. doi:10.1109/VLHCC.2005.70
- [5] Eirik Bakke, David Karger, and Rob Miller. 2011. A spreadsheet-based user interface for managing plural relationships in structured data. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Vancouver BC Canada, 2541–2550. doi:10.1145/1978942.1979313
- [6] Eirik Bakke and David R. Karger. 2016. Expressive Query Construction through Direct Manipulation of Nested Relational Results. In *Proceedings of the 2016 International Conference on Management of Data*. ACM, San Francisco California USA, 1377–1392. doi:10.1145/2882903.2915210
- [7] Alan F. Blackwell, Carol Britton, Anna Louise Cox, Thomas R. G. Green, Corin A. Gurr, Gada F. Kadoda, Maria Kutar, Martin Loomes, Chrystopher L. Nehaniv, Marian Petre, Chris Roast, Chris Roe, Allan Wong, and Richard M. Young. 2001. Cognitive Dimensions of Notations: Design Tools for Cognitive Technology. In *Proceedings of the 4th International Conference on Cognitive Technology: Instruments of Mind (CT '01)*. Springer-Verlag, Berlin, Heidelberg, 325–341.
- [8] Polly S. Brown and John D. Gould. 1987. An experimental study of people creating spreadsheets. *ACM Trans. Inf. Syst.* 5, 3 (July 1987), 258–272. doi:10.1145/27641.28058
- [9] Margaret Burnett, John Atwood, Rebecca Walpole Djang, James Reichwein, Herkimer Gottfried, and Sherry Yang. 2001. Forms/3: A first-order visual language to explore the boundaries of the spreadsheet paradigm. *J. Funct. Program.* 11, 2 (March 2001), 155–206.
- [10] George Chalhoub and Advait Sarkar. 2022. "It's Freedom to Put Things Where My Mind Wants": Understanding and Improving the User Experience of Structuring Data in Spreadsheets. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–24. doi:10.1145/3491102.3501833
- [11] Kerry Shih-Ping Chang and Brad A. Myers. 2014. A spreadsheet model for using web service data. In *2014 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 169–176. doi:10.1109/VLHCC.2014.6883042 ISSN: 1943-6106.
- [12] Kerry Shih-Ping Chang and Brad A. Myers. 2016. Using and Exploring Hierarchical Data in Spreadsheets. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (CHI '16)*. Association for Computing Machinery, New York, NY, USA, 2497–2507. doi:10.1145/2858036.2858430
- [13] Jácume Cunha, João P. Fernandes, Jorge Mendes, Hugo Pacheco, and João Saraiva. 2012. Bidirectional Transformation of Model-Driven Spreadsheets. In *Theory and Practice of Model Transformations*, Zhenjiang Hu and Juan de Lara (Eds.). Springer, Berlin, Heidelberg, 105–120. doi:10.1007/978-3-642-30476-7_7
- [14] Jácume Cunha, João Paulo Fernandes, Jorge Mendes, and João Saraiva. 2012. MDSheet: A framework for model-driven spreadsheet engineering. In *2012 34th International Conference on Software Engineering (ICSE)*. 1395–1398. doi:10.1109/ICSE.2012.6227239
- [15] Jácume Cunha, Jorge Mendes, João Saraiva, and Joost Visser. 2014. Model-based programming environments for spreadsheets. *Science of Computer Programming* 96 (Dec. 2014), 254–275. doi:10.1016/j.scico.2014.02.002
- [16] Gregor Engels and Martin Erwig. 2005. ClassSheets: automatic generation of spreadsheet applications from object-oriented specifications. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering*. ACM, Long Beach CA USA, 124–133. doi:10.1145/1101908.1101929
- [17] Martin Erwig, Robin Abraham, Irene Cooperstein, and Steve Kollmansberger. 2005. Automatic generation and maintenance of correct spreadsheets. In *Proceedings of the 27th international conference on Software engineering (ICSE '05)*. Association for Computing Machinery, New York, NY, USA, 136–145. doi:10.1145/1062455.1062494
- [18] Martin Erwig, Robin Abraham, Steve Kollmansberger, and Irene Cooperstein. 2006. Gencil: a program generator for correct spreadsheets. *Journal of Functional Programming* 16, 03 (May 2006), 293. doi:10.1017/S0956796805005794
- [19] Marc Fisher and Gregg Rothermel. 2005. The EUSES spreadsheet corpus: a shared resource for supporting experimentation with spreadsheet dependability mechanisms. In *Proceedings of the first workshop on End-user software engineering (WEUSE I)*. Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/1083231.1083242
- [20] Alan Hall, Michel Wermelinger, Tony Hirst, and Santi Phithakkittukoon. 2018. Structuring Spreadsheets with the "Lish" Data Model. doi:10.48550/arXiv.1801.08603 arXiv:1801.08603 [cs].
- [21] Nima Joharizadeh, Advait Sarkar, Andrew D. Gordon, and Jack Williams. 2020. Gridlets: Reusing Spreadsheet Grids. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, Honolulu HI USA, 1–7. doi:10.1145/3334480.3382806
- [22] Elvis Koci, Maik Thiele, Oscar Romero, and Wolfgang Lehner. 2016. A Machine Learning Approach for Layout Inference in Spreadsheets, Vol. 2. SCITEPRESS, 77–88. doi:10.5220/0006052200770088
- [23] Elvis Koci, Maik Thiele, Oscar Romero, and Wolfgang Lehner. 2017. Table Identification and Reconstruction in Spreadsheets. In *Advanced Information Systems Engineering*, Eric Dubois and Klaus Pohl (Eds.). Springer International Publishing, Cham, 527–541. doi:10.1007/978-3-319-59536-8_33
- [24] Jenny T. Liang, Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Vu Le, Chris Parnin, Arjun Radhakrishna, Ashish Tiwari, Emerson Murphy-Hill, and Gustavo Soares. 2025. TableTalk: Scaffolding Spreadsheet Development with a Language Agent. *ACM Trans. Comput.-Hum. Interact.* 32, 6 (Dec. 2025), 65:1–65:49. doi:10.1145/3765286
- [25] Matt McCutchen, Shachar Itzhaky, and Daniel Jackson. 2016. Object spreadsheets: a new computational model for end-user development of data-centric web applications. In *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, Amsterdam Netherlands, 112–127. doi:10.1145/2986012.2986018
- [26] Darren Miller, Gary Miller, and Luis M. Parrondo. 2010. Sumwise: A smarter spreadsheet. In *EuSpRiG*. https://www.researchgate.net/profile/Gary-Miller-11/publication/267803810_Sumwise_A_Smarter_Spreadsheet/links/55f0b28a08ae199d47c22057/Sumwise-A-Smarter-Spreadsheet.pdf
- [27] Gary Miller and Felienne Hermans. 2016. Gradual structuring in the spreadsheet paradigm. In *2016 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 240–241. doi:10.1109/VLHCC.2016.7739696 ISSN: 1943-6106.
- [28] Gary Miller, Felienne Hermans, and Robin Braun. 2016. Gradual structuring: Evolving the spreadsheet paradigm for expressiveness and learnability. In *2016 15th International Conference on Information Technology Based Higher Education and Training (ITHET)*. 1–8. doi:10.1109/ITHET.2016.7760759
- [29] R. Mittermeir and M. Clermont. 2002. Finding high-level structures in spreadsheet programs. In *Ninth Working Conference on Reverse Engineering, 2002. Proceedings.* 221–232. doi:10.1109/WCRE.2002.1173080
- [30] Bonnie A. Nardi. 1993. *A small matter of programming: perspectives on end user computing*. MIT Press, Cambridge, MA.
- [31] Jocelyn Paine. 2008. Ensuring Spreadsheet Integrity with Model Master. doi:10.48550/arXiv.0801.3690 arXiv:0801.3690 [cs].
- [32] R.R. Panko and R.P. Halverson. 1996. Spreadsheets on trial: a survey of research on spreadsheet risks. In *Proceedings of HICSS-29: 29th Hawaii International Conference on System Sciences*, Vol. 2. 326–335 vol.2. doi:10.1109/HICSS.1996.495416
- [33] Raymond R. Panko. 1998. What We Know About Spreadsheet Errors. *Journal of Organizational and End User Computing (JOEUC)* 10, 2 (1998), 15–21. doi:10.4018/joeuc.1998040102
- [34] Raymond R. Panko. 2008. Spreadsheet Errors: What We Know. What We Think We Can Do. doi:10.48550/arXiv.0802.3457 arXiv:0802.3457 [cs].
- [35] Stephen G. Powell, Kenneth R. Baker, and Barry Lawson. 2008. A critical review of the literature on spreadsheet errors. *Decision Support Systems* 46, 1 (Dec. 2008), 128–138. doi:10.1016/j.dss.2008.06.001
- [36] C. Scaffidi, M. Shaw, and B. Myers. 2005. Estimating the numbers of end users and end user programmers. In *2005 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC'05)*. 207–214. doi:10.1109/VLHCC.2005.34 ISSN: 1943-6106.
- [37] Ricardo Teixeira and Vasco Amaral. 2016. On the Emergence of Patterns for Spreadsheets Data Arrangements. In *Software Technologies: Applications and Foundations*, Paolo Milazzo, Dániel Varró, and Manuel Wimmer (Eds.). Vol. 9946. Springer International Publishing, Cham, 333–345. doi:10.1007/978-3-319-50230-4_25 Series Title: Lecture Notes in Computer Science.
- [38] Yakun Zhang, Xiao Lv, Haoyu Dong, Wensheng Dou, Shi Han, Dongmei Zhang, Jun Wei, and Dan Ye. 2021. Semantic table structure identification in spreadsheets. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 283–295. doi:10.1145/3460319.3464812

A Expressiveness Study Examples

Repeated patterns, non-tabular groupings, nested structure, and labels on the grid.

The left spreadsheet shows a grid with columns for dates, times, and attendance status. It includes a table for 'Edison Middle School Attendance Calculation Sheet (8th Grade)' and another for 'Edison Middle School Attendance Calculation Sheet (7th Grade)'. The right spreadsheet shows a grid with columns for years, months, and financial data. It includes a table for 'Cuenta Financiera del Sector Público (Millones de U.S. dólares) 1/ 2/ 3/ 4/ 5/ 6/ 7/ 8/ 9/ 10/ 11/ 12/ 13/ 14/ 15/ 16/ 17/ 18/ 19/ 20/ 21/ 22/ 23/ 24/ 25/ 26/ 27/ 28/ 29/ 30/ 31/ 32/ 33/ 34/ 35/ 36/ 37/ 38/ 39/ 40/ 41/ 42/ 43/ 44/ 45/ 46/ 47/ 48/ 49/ 50/ 51/ 52/ 53/ 54/ 55/ 56/ 57/ 58/ 59/ 60/ 61/ 62/ 63/ 64/ 65/ 66/ 67/ 68/ 69/ 70/ 71/ 72/ 73/ 74/ 75/ 76/ 77/ 78/ 79/ 80/ 81/ 82/ 83/ 84/ 85/ 86/ 87/ 88/ 89/ 90/ 91/ 92/ 93/ 94/ 95/ 96/ 97/ 98/ 99/ 100/ 101/ 102/ 103/ 104/ 105/ 106/ 107/ 108/ 109/ 110/ 111/ 112/ 113/ 114/ 115/ 116/ 117/ 118/ 119/ 120/ 121/ 122/ 123/ 124/ 125/ 126/ 127/ 128/ 129/ 130/ 131/ 132/ 133/ 134/ 135/ 136/ 137/ 138/ 139/ 140/ 141/ 142/ 143/ 144/ 145/ 146/ 147/ 148/ 149/ 150/ 151/ 152/ 153/ 154/ 155/ 156/ 157/ 158/ 159/ 160/ 161/ 162/ 163/ 164/ 165/ 166/ 167/ 168/ 169/ 170/ 171/ 172/ 173/ 174/ 175/ 176/ 177/ 178/ 179/ 180/ 181/ 182/ 183/ 184/ 185/ 186/ 187/ 188/ 189/ 190/ 191/ 192/ 193/ 194/ 195/ 196/ 197/ 198/ 199/ 200/ 201/ 202/ 203/ 204/ 205/ 206/ 207/ 208/ 209/ 210/ 211/ 212/ 213/ 214/ 215/ 216/ 217/ 218/ 219/ 220/ 221/ 222/ 223/ 224/ 225/ 226/ 227/ 228/ 229/ 230/ 231/ 232/ 233/ 234/ 235/ 236/ 237/ 238/ 239/ 240/ 241/ 242/ 243/ 244/ 245/ 246/ 247/ 248/ 249/ 250/ 251/ 252/ 253/ 254/ 255/ 256/ 257/ 258/ 259/ 260/ 261/ 262/ 263/ 264/ 265/ 266/ 267/ 268/ 269/ 270/ 271/ 272/ 273/ 274/ 275/ 276/ 277/ 278/ 279/ 280/ 281/ 282/ 283/ 284/ 285/ 286/ 287/ 288/ 289/ 290/ 291/ 292/ 293/ 294/ 295/ 296/ 297/ 298/ 299/ 300/ 301/ 302/ 303/ 304/ 305/ 306/ 307/ 308/ 309/ 310/ 311/ 312/ 313/ 314/ 315/ 316/ 317/ 318/ 319/ 320/ 321/ 322/ 323/ 324/ 325/ 326/ 327/ 328/ 329/ 330/ 331/ 332/ 333/ 334/ 335/ 336/ 337/ 338/ 339/ 340/ 341/ 342/ 343/ 344/ 345/ 346/ 347/ 348/ 349/ 350/ 351/ 352/ 353/ 354/ 355/ 356/ 357/ 358/ 359/ 360/ 361/ 362/ 363/ 364/ 365/ 366/ 367/ 368/ 369/ 370/ 371/ 372/ 373/ 374/ 375/ 376/ 377/ 378/ 379/ 380/ 381/ 382/ 383/ 384/ 385/ 386/ 387/ 388/ 389/ 390/ 391/ 392/ 393/ 394/ 395/ 396/ 397/ 398/ 399/ 400/ 401/ 402/ 403/ 404/ 405/ 406/ 407/ 408/ 409/ 410/ 411/ 412/ 413/ 414/ 415/ 416/ 417/ 418/ 419/ 420/ 421/ 422/ 423/ 424/ 425/ 426/ 427/ 428/ 429/ 430/ 431/ 432/ 433/ 434/ 435/ 436/ 437/ 438/ 439/ 440/ 441/ 442/ 443/ 444/ 445/ 446/ 447/ 448/ 449/ 450/ 451/ 452/ 453/ 454/ 455/ 456/ 457/ 458/ 459/ 460/ 461/ 462/ 463/ 464/ 465/ 466/ 467/ 468/ 469/ 470/ 471/ 472/ 473/ 474/ 475/ 476/ 477/ 478/ 479/ 480/ 481/ 482/ 483/ 484/ 485/ 486/ 487/ 488/ 489/ 490/ 491/ 492/ 493/ 494/ 495/ 496/ 497/ 498/ 499/ 500/ 501/ 502/ 503/ 504/ 505/ 506/ 507/ 508/ 509/ 510/ 511/ 512/ 513/ 514/ 515/ 516/ 517/ 518/ 519/ 520/ 521/ 522/ 523/ 524/ 525/ 526/ 527/ 528/ 529/ 530/ 531/ 532/ 533/ 534/ 535/ 536/ 537/ 538/ 539/ 540/ 541/ 542/ 543/ 544/ 545/ 546/ 547/ 548/ 549/ 550/ 551/ 552/ 553/ 554/ 555/ 556/ 557/ 558/ 559/ 560/ 561/ 562/ 563/ 564/ 565/ 566/ 567/ 568/ 569/ 570/ 571/ 572/ 573/ 574/ 575/ 576/ 577/ 578/ 579/ 580/ 581/ 582/ 583/ 584/ 585/ 586/ 587/ 588/ 589/ 590/ 591/ 592/ 593/ 594/ 595/ 596/ 597/ 598/ 599/ 600/ 601/ 602/ 603/ 604/ 605/ 606/ 607/ 608/ 609/ 610/ 611/ 612/ 613/ 614/ 615/ 616/ 617/ 618/ 619/ 620/ 621/ 622/ 623/ 624/ 625/ 626/ 627/ 628/ 629/ 630/ 631/ 632/ 633/ 634/ 635/ 636/ 637/ 638/ 639/ 640/ 641/ 642/ 643/ 644/ 645/ 646/ 647/ 648/ 649/ 650/ 651/ 652/ 653/ 654/ 655/ 656/ 657/ 658/ 659/ 660/ 661/ 662/ 663/ 664/ 665/ 666/ 667/ 668/ 669/ 670/ 671/ 672/ 673/ 674/ 675/ 676/ 677/ 678/ 679/ 680/ 681/ 682/ 683/ 684/ 685/ 686/ 687/ 688/ 689/ 690/ 691/ 692/ 693/ 694/ 695/ 696/ 697/ 698/ 699/ 700/ 701/ 702/ 703/ 704/ 705/ 706/ 707/ 708/ 709/ 710/ 711/ 712/ 713/ 714/ 715/ 716/ 717/ 718/ 719/ 720/ 721/ 722/ 723/ 724/ 725/ 726/ 727/ 728/ 729/ 730/ 731/ 732/ 733/ 734/ 735/ 736/ 737/ 738/ 739/ 740/ 741/ 742/ 743/ 744/ 745/ 746/ 747/ 748/ 749/ 750/ 751/ 752/ 753/ 754/ 755/ 756/ 757/ 758/ 759/ 760/ 761/ 762/ 763/ 764/ 765/ 766/ 767/ 768/ 769/ 770/ 771/ 772/ 773/ 774/ 775/ 776/ 777/ 778/ 779/ 780/ 781/ 782/ 783/ 784/ 785/ 786/ 787/ 788/ 789/ 790/ 791/ 792/ 793/ 794/ 795/ 796/ 797/ 798/ 799/ 800/ 801/ 802/ 803/ 804/ 805/ 806/ 807/ 808/ 809/ 810/ 811/ 812/ 813/ 814/ 815/ 816/ 817/ 818/ 819/ 820/ 821/ 822/ 823/ 824/ 825/ 826/ 827/ 828/ 829/ 830/ 831/ 832/ 833/ 834/ 835/ 836/ 837/ 838/ 839/ 840/ 841/ 842/ 843/ 844/ 845/ 846/ 847/ 848/ 849/ 850/ 851/ 852/ 853/ 854/ 855/ 856/ 857/ 858/ 859/ 860/ 861/ 862/ 863/ 864/ 865/ 866/ 867/ 868/ 869/ 870/ 871/ 872/ 873/ 874/ 875/ 876/ 877/ 878/ 879/ 880/ 881/ 882/ 883/ 884/ 885/ 886/ 887/ 888/ 889/ 890/ 891/ 892/ 893/ 894/ 895/ 896/ 897/ 898/ 899/ 900/ 901/ 902/ 903/ 904/ 905/ 906/ 907/ 908/ 909/ 910/ 911/ 912/ 913/ 914/ 915/ 916/ 917/ 918/ 919/ 920/ 921/ 922/ 923/ 924/ 925/ 926/ 927/ 928/ 929/ 930/ 931/ 932/ 933/ 934/ 935/ 936/ 937/ 938/ 939/ 940/ 941/ 942/ 943/ 944/ 945/ 946/ 947/ 948/ 949/ 950/ 951/ 952/ 953/ 954/ 955/ 956/ 957/ 958/ 959/ 960/ 961/ 962/ 963/ 964/ 965/ 966/ 967/ 968/ 969/ 970/ 971/ 972/ 973/ 974/ 975/ 976/ 977/ 978/ 979/ 980/ 981/ 982/ 983/ 984/ 985/ 986/ 987/ 988/ 989/ 990/ 991/ 992/ 993/ 994/ 995/ 996/ 997/ 998/ 999/ 1000/ 1001/ 1002/ 1003/ 1004/ 1005/ 1006/ 1007/ 1008/ 1009/ 1010/ 1011/ 1012/ 1013/ 1014/ 1015/ 1016/ 1017/ 1018/ 1019/ 1020/ 1021/ 1022/ 1023/ 1024/ 1025/ 1026/ 1027/ 1028/ 1029/ 1030/ 1031/ 1032/ 1033/ 1034/ 1035/ 1036/ 1037/ 1038/ 1039/ 1040/ 1041/ 1042/ 1043/ 1044/ 1045/ 1046/ 1047/ 1048/ 1049/ 1050/ 1051/ 1052/ 1053/ 1054/ 1055/ 1056/ 1057/ 1058/ 1059/ 1060/ 1061/ 1062/ 1063/ 1064/ 1065/ 1066/ 1067/ 1068/ 1069/ 1070/ 1071/ 1072/ 1073/ 1074/ 1075/ 1076/ 1077/ 1078/ 1079/ 1080/ 1081/ 1082/ 1083/ 1084/ 1085/ 1086/ 1087/ 1088/ 1089/ 1090/ 1091/ 1092/ 1093/ 1094/ 1095/ 1096/ 1097/ 1098/ 1099/ 1100/ 1101/ 1102/ 1103/ 1104/ 1105/ 1106/ 1107/ 1108/ 1109/ 1110/ 1111/ 1112/ 1113/ 1114/ 1115/ 1116/ 1117/ 1118/ 1119/ 1120/ 1121/ 1122/ 1123/ 1124/ 1125/ 1126/ 1127/ 1128/ 1129/ 1130/ 1131/ 1132/ 1133/ 1134/ 1135/ 1136/ 1137/ 1138/ 1139/ 1140/ 1141/ 1142/ 1143/ 1144/ 1145/ 1146/ 1147/ 1148/ 1149/ 1150/ 1151/ 1152/ 1153/ 1154/ 1155/ 1156/ 1157/ 1158/ 1159/ 1160/ 1161/ 1162/ 1163/ 1164/ 1165/ 1166/ 1167/ 1168/ 1169/ 1170/ 1171/ 1172/ 1173/ 1174/ 1175/ 1176/ 1177/ 1178/ 1179/ 1180/ 1181/ 1182/ 1183/ 1184/ 1185/ 1186/ 1187/ 1188/ 1189/ 1190/ 1191/ 1192/ 1193/ 1194/ 1195/ 1196/ 1197/ 1198/ 1199/ 1200/ 1201/ 1202/ 1203/ 1204/ 1205/ 1206/ 1207/ 1208/ 1209/ 1210/ 1211/ 1212/ 1213/ 1214/ 1215/ 1216/ 1217/ 1218/ 1219/ 1220/ 1221/ 1222/ 1223/ 1224/ 1225/ 1226/ 1227/ 1228/ 1229/ 1230/ 1231/ 1232/ 1233/ 1234/ 1235/ 1236/ 1237/ 1238/ 1239/ 1240/ 1241/ 1242/ 1243/ 1244/ 1245/ 1246/ 1247/ 1248/ 1249/ 1250/ 1251/ 1252/ 1253/ 1254/ 1255/ 1256/ 1257/ 1258/ 1259/ 1260/ 1261/ 1262/ 1263/ 1264/ 1265/ 1266/ 1267/ 1268/ 1269/ 1270/ 1271/ 1272/ 1273/ 1274/ 1275/ 1276/ 1277/ 1278/ 1279/ 1280/ 1281/ 1282/ 1283/ 1284/ 1285/ 1286/ 1287/ 1288/ 1289/ 1290/ 1291/ 1292/ 1293/ 1294/ 1295/ 1296/ 1297/ 1298/ 1299/ 1300/ 1301/ 1302/ 1303/ 1304/ 1305/ 1306/ 1307/ 1308/ 1309/ 1310/ 1311/ 1312/ 1313/ 1314/ 1315/ 1316/ 1317/ 1318/ 1319/ 1320/ 1321/ 1322/ 1323/ 1324/ 1325/ 1326/ 1327/ 1328/ 1329/ 1330/ 1331/ 1332/ 1333/ 1334/ 1335/ 1336/ 1337/ 1338/ 1339/ 1340/ 1341/ 1342/ 1343/ 1344/ 1345/ 1346/ 1347/ 1348/ 1349/ 1350/ 1351/ 1352/ 1353/ 1354/ 1355/ 1356/ 1357/ 1358/ 1359/ 1360/ 1361/ 1362/ 1363/ 1364/ 1365/ 1366/ 1367/ 1368/ 1369/ 1370/ 1371/ 1372/ 1373/ 1374/ 1375/ 1376/ 1377/ 1378/ 1379/ 1380/ 1381/ 1382/ 1383/ 1384/ 1385/ 1386/ 1387/ 1388/ 1389/ 1390/ 1391/ 1392/ 1393/ 1394/ 1395/ 1396/ 1397/ 1398/ 1399/ 1400/ 1401/ 1402/ 1403/ 1404/ 1405/ 1406/ 1407/ 1408/ 1409/ 1410/ 1411/ 1412/ 1413/ 1414/ 1415/ 1416/ 1417/ 1418/ 1419/ 1420/ 1421/ 1422/ 1423/ 1424/ 1425/ 1426/ 1427/ 1428/ 1429/ 1430/ 1431/ 1432/ 1433/ 1434/ 1435/ 1436/ 1437/ 1438/ 1439/ 1440/ 1441/ 1442/ 1443/ 1444/ 1445/ 1446/ 1447/ 1448/ 1449/ 1450/ 1451/ 1452/ 1453/ 1454/ 1455/ 1456/ 1457/ 1458/ 1459/ 1460/ 1461/ 1462/ 1463/ 1464/ 1465/ 1466/ 1467/ 1468/ 1469/ 1470/ 1471/ 1472/ 1473/ 1474/ 1475/ 1476/ 1477/ 1478/ 1479/ 1480/ 1481/ 1482/ 1483/ 1484/ 1485/ 1486/ 1487/ 1488/ 1489/ 1490/ 1491/ 1492/ 1493/ 1494/ 1495/ 1496/ 1497/ 1498/ 1499/ 1500/ 1501/ 1502/ 1503/ 1504/ 1505/ 1506/ 1507/ 1508/ 1509/ 1510/ 1511/ 1512/ 1513/ 1514/ 1515/ 1516/ 1517/ 1518/ 1519/ 1520/ 1521/ 1522/ 1523/ 1524/ 1525/ 1526/ 1527/ 1528/ 1529/ 1530/ 1531/ 1532/ 1533/ 1534/ 1535/ 1536/ 1537/ 1538/ 1539/ 1540/ 1541/ 1542/ 1543/ 1544/ 1545/ 1546/ 1547/ 1548/ 1549/ 1550/ 1551/ 1552/ 1553/ 1554/ 1555/ 1556/ 1557/ 1558/ 1559/ 1560/ 1561/ 1562/ 1563/ 1564/ 1565/ 1566/ 1567/ 1568/ 1569/ 1570/ 1571/ 1572/ 1573/ 1574/ 1575/ 1576/ 1577/ 1578/ 1579/ 1580/ 1581/ 1582/ 1583/ 1584/ 1585/ 1586/ 1587/ 1588/ 1589/ 1590/ 1591/ 1592/ 1593/ 1594/ 1595/ 1596/ 1597/ 1598/ 1599/ 1600/ 1601/ 1602/ 1603/ 1604/ 1605/ 1606/ 1607/ 1608/ 1609/ 1610/ 1611/ 1612/ 1613/ 1614/ 1615/ 1616/ 1617/ 1618/ 1619/ 1620/ 1621/ 1622/ 1623/ 1624/ 1625/ 1626/ 1627/ 1628/ 1629/ 1630/ 1631/ 1632/ 1633/ 1634/ 1635/ 1636/ 1637/ 1638/ 1639/ 1640/ 1641/ 1642/ 1643/ 1644/ 1645/ 1646/ 1647/ 1648/ 1649/ 1650/ 1651/ 1652/ 1653/ 1654/ 1655/ 1656/ 1657/ 1658/ 1659/ 1660/ 1661/ 1662/ 1663/ 1664/ 1665/ 1666/ 1667/ 1668/ 1669/ 1670/ 1671/ 1672/ 1673/ 1674/ 1675/ 1676/ 1677/ 1678/ 1679/ 1680/ 1681/ 1682/ 1683/ 1684/ 1685/ 1686/ 1687/ 1688/ 1689/ 1690/ 1691/ 1692/ 1693/ 1694/ 1695/ 1696/ 1697/ 1698/ 1699/ 1700/ 1701/ 1702/ 1703/ 1704/ 1705/ 1706/ 1707/ 1708/ 1709/ 1710/ 1711/ 1712/ 1713/ 1714/ 1715/ 1716/ 1717/ 1718/ 1719/ 1720/ 1721/ 1722/ 1723/ 1724/ 1725/ 1726/ 1727/ 1728/ 1729/ 1730/ 1731/ 1732/ 1733/ 1734/ 1735/ 1736/ 1737/ 1738/ 1739/ 1740/ 1741/ 1742/ 1743/ 1744/ 1745/ 1746/ 1747/ 1748/ 1749/ 1750/ 1751/ 1752/ 1753/ 1754/ 1755/ 1756/ 1757/ 1758/ 1759/ 1760/ 1761/ 1762/ 1763/ 1764/ 1765/ 1766/ 1767/ 1768/ 1769/ 1770/ 1771/ 1772/ 1773/ 1774/ 1775/ 1776/ 1777/ 1778/ 1779/ 1780/ 1781/ 1782/ 1783/ 1784/ 1785/ 1786/ 1787/ 1788/ 1789/ 1790/ 1791/ 1792/ 1793/ 1794/ 1795/ 1796/ 1797/ 1798/ 1799/ 1800/ 1801/ 1802/ 1803/ 1804/ 1805/ 1806/ 1807/ 1808/ 1809/ 1810/ 1811/ 1812/ 1813/ 1814/ 1815/ 1816/ 1817/ 1818/ 1819/ 1820/ 1821/ 1822/ 1823/ 1824/ 1825/ 1826/ 1827/ 1828/ 1829/ 1830/ 1831/ 1832/ 1833/ 1834/ 1835/ 1836/ 1837/ 1838/ 1839/ 1840/ 1841/ 1842/ 1843/ 1844/ 1845/ 1846/ 1847/ 1848/ 1849/ 1850/ 1851/ 1852/ 1853/ 1854/ 1855/ 1856/ 1857/ 1858/ 1859/ 1860/ 1861/ 1862/ 1863/ 1864/ 1865/ 1866/ 1867/ 1868/ 1869/ 1870/ 1871/ 1872/ 1873/ 1874/ 1875/ 1876/ 1877/ 1878/ 1879/ 1880/ 1881/ 1882/ 1883/ 1884/ 1885/ 1886/ 1887/ 1888/ 1889/ 1890/ 1891/ 1892/ 1893/ 1894/ 1895/ 1896/ 1897/ 1898/ 1899/ 1900/ 1901/ 1902/ 1903/ 1904/ 1905/ 1906/ 1907/ 1908/ 1909/ 1910/ 1911/ 1912/ 1913/ 1914/ 1915/ 1916/ 1917/ 1918/ 1919/ 1920/ 1921/ 1922/ 1923/ 1924/ 1925/ 1926/ 1927/ 1928/ 1929/ 1930/ 1931/ 1932/ 1933/ 1934/ 1935/ 1936/ 1937/ 1938/ 1939/ 1940/ 1941/ 1942/ 1943/ 1944/ 1945/ 1946/ 1947/ 1948/ 1949/ 1950/ 1951/ 1952/ 1953/ 1954/ 1955/ 1956/ 1957/ 1958/ 1959/ 1960/ 1961/ 1962/ 1963/ 1964/ 1965/ 1966/ 1967/ 1968/ 1969/ 1970/ 1971/ 1972/ 1973/ 1974/ 1975/ 1976/ 1977/ 1978/ 1979/ 1980/ 1981/ 1982/ 1983/ 1984/ 1985/ 1986/ 1987/ 1988/ 1989/ 1990/ 1991/ 1992/ 1993/ 1994/ 1995/ 1996/ 1997/ 1998/ 1999/ 2000/ 2001/ 2002/ 2003/ 2004/ 2005/ 2006/ 2007/ 2008/ 2009/ 2010/ 2011/ 2012/ 2013/ 2014/ 2015/ 2016/ 2017/ 2018/ 2019/ 2020/ 2021/ 2022/ 2023/ 2024/ 2025/ 2026/ 2027/ 2028/ 2029/ 2030/ 2031/ 2032/ 2033/ 2034/ 2035/ 2036/ 2037/ 2038/ 2039/ 2040/ 2041/ 2042/ 2043/ 2044/ 2045/ 2046/ 2047/ 2048/ 2049/ 2050/ 2051/ 2052/ 2053/ 2054/ 2055/ 2056/ 2057/ 2058/ 2059/ 2060/ 2061/ 2062/ 2063/ 2064/ 2065/ 2066/ 2067/ 2068/ 2069/ 2070/ 2071/ 2072/ 2073/ 2074/ 2075/ 2076/ 2077/ 2078/ 2079/ 2080/ 2081/ 2082/ 2083/ 2084/ 2085/ 2086/ 2087/ 2088/ 2089/ 2090/ 2091/ 2092/ 2093/ 2094/ 2095/ 2096/ 2097/ 2098/ 2099/ 2100/ 2101/ 2102/ 2103/ 2104/ 2105/ 2106/ 2107/ 2108/ 2109/ 2110/ 2111/ 2112/ 2113/ 2114/ 2115/ 2116/ 2117/ 2118/ 2119/ 2120/ 2121/ 2122/ 2123/ 2124/ 2125/ 2126/ 2127/ 2128/ 2129/ 2130/ 2131/ 2132/ 2133/ 2134/ 2135/ 2136/ 2137/ 2138/ 2139/ 2140/ 2141/ 2142/ 2143/ 2144/ 2145/ 2146/ 2147/ 2148/ 2149/ 2150/ 2151/ 2152/ 2153/ 2154/ 2155/ 2156/ 2157/ 2158/ 2159/ 2160/ 2161/ 2162/ 2163/ 2164/ 2165/ 2166/ 2167/ 2168/ 2169/ 2170/ 2171/ 2172/ 2173/ 2174/ 2175/ 2176/ 2177/ 2178/ 2179/ 2180/ 218

Nested structure and labels on the grid.

Faculty of Arts and Sciences Workflow Matrix by Type of Appointment								
Type of Appointment	Object Code	Who initiates action in Asperin	Who approves** action in Asperin after receiving backup paperwork	Who Prints PS form and sends with backup paperwork to Holyoke Ctr.***	Payroll Paid From and Frequency	Payroll Date	Who Approves Extra Comp. Pymts. (Note A)	Uses Psoft Time and Labor System?
Senior Faculty*	6010	Academic Affairs	Academic Affairs	Academic Affairs	Faculty - Monthly	Last day of Month	FAS Financial Office	No
Junior Faculty	6020	Department	Academic Affairs	Academic Affairs	Faculty - Monthly	Last day of Month	FAS Financial Office	No
Other Teaching & Research Appts.	6030	Department	Academic Affairs Div. of Cont. Ed.	Academic Affairs Div. of Cont. Ed.	Faculty - Monthly	Last day of Month	FAS Financial Office Div. of Cont. Ed.	No
Exempt Staff*	6050	Department	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	Staff - Biweekly	Every other Friday	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	No (eventually for paid time off)
Overtime Eligible Staff	6050 6070	Department	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	Staff - Biweekly	Every other Friday	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	Yes



Faculty of Arts and Sciences Workflow Matrix by Type of Appointment											
Type of Appointment	Object Code	Who initiates action in Asperin	System Used for Personnel Actions	Who approves** action in Asperin after receiving backup paperwork	Who Prints PS form and sends with backup paperwork to Holyoke Ctr.***	Payroll Paid From and Frequency	Payroll Frequency	Payroll Date	Who Approves Extra Comp. Pymts. (Note A)	Uses Psoft Time and Labor System?	Dept. Responsible for Payroll
Senior Faculty*	6010	Academic Affairs	Asperin	Academic Affairs	Academic Affairs	Faculty - Monthly	Monthly	Last day of Month	FAS Financial Office	No	FAS Payroll Office
Junior Faculty	6020	Department	Asperin	Academic Affairs	Academic Affairs	Faculty - Monthly	Monthly	Last day of Month	FAS Financial Office	No	FAS Payroll Office
Other Teaching & Research Appts.	6030	Department	Asperin	Academic Affairs Div. of Cont. Ed.	Academic Affairs Div. of Cont. Ed.	Faculty - Monthly	Monthly	Last day of Month	FAS Financial Office Div. of Cont. Ed.	No	FAS Payroll Office
Exempt Staff*	6050	Department	Asperin	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	Staff - Biweekly	Semi-Monthly	Every other Friday	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	No (eventually for paid time off)	FAS Payroll Office
Overtime Eligible Staff	6050 6070	Department	Asperin	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	Staff - Biweekly	Semi-Monthly	Every other Friday	FAS Personnel Svcs Hvd College Library Div. of Cont. Ed.	Yes	FAS Payroll Office

Figure 9: Example of a reproduced spreadsheet from our expressiveness study.

B User Study Materials

B.1 Task 1

Part 1: You are a graduate student. Your research group has weekly meetings, and you want to record attendance at each week's meetings this quarter. Create a spreadsheet that, for each meeting, records:

- the week number
- the topic of the meeting
- the names of attendees
- the count of attendees

Make sure to design your spreadsheet so that new meetings and attendees can easily be added.

Data:

- Week 1, "Literature Review"; Attendees: Alice, Bob
- Week 2, "Experiment Design"; Attendees: Alice, Carol, David

- Week 3, “Related Work Discussion”; Attendees: Bob, Carol, Eve

Part 2: Add two new meetings to the meeting record.

Data:

- Week 4, “Pilot Study Results”; Attendees: Alice, Bob, David, Grace
- Week 5, “Writing Workshop”; Attendees: Alice, Frank

Part 3: Using your spreadsheet, compute the following:

- The average number of attendees across all meetings this quarter
- A list of the topics covered this quarter

B.2 Task 2

Part 1: You are a teacher who taught a course in geometry to high school students. Your job is to compute the overall grade of each student in the class, as well as the average grade across the whole class. Each student’s final grade is a weighted average of 3 types of assignments: homeworks, exams, and participation activity. In your course, there were 2 homeworks, 2 exams, and 1 participation activity. The assignments are worth 40% of the grade, the exams are worth 50% of the grade, and the participation activity is worth 10% of the grade. All grades should be expressed as percentages. Create a spreadsheet computing the overall grade for each student in the class, as well as the average grade across the whole class. Make sure to design the spreadsheet so that new data can be added as necessary.

Data:

- Maria: Homework 1, 90; Homework 2, 93; Exam 1, 94; Exam 2, 89; Participation, 95.
- David: Homework 1, 65; Homework 2, 70; Exam 1, 75; Exam 2, 73; Participation, 80.
- Aisha: Homework 1, 88; Homework 2, 84; Exam 1, 92; Exam 2, 87; Participation, 93.

Part 2: You’ve finished grading assignments for a couple new students, and need to add them to the sheet. Add these students, making sure to compute their overall grade and update the class average accordingly.

Data:

- Ethan: Homework 1, 78; Homework 2, 82; Exam 1, 88; Exam 2, 90; Participation, 87.
- Sofia: Homework 1, 95; Homework 2, 92; Exam 1, 91; Exam 2, 93; Participation, 98.

Part 3: It’s a new semester, and now you have a different class of students. Also, this semester there is one more homework assignment (for a total of 3). Produce a new section in the spreadsheet for Semester 2 using the same grade calculation. As before, you should also compute the average for the new class.

Data:

- Priya: Homework 1, 83; Homework 2, 79; Homework 3, 85; Exam 1, 76; Exam 2, 80; Participation, 88.
- Chi: Homework 1, 91; Homework 2, 89; Homework 3, 92; Exam 1, 90; Exam 2, 93; Participation, 97.
- Noah: Homework 1, 74; Homework 2, 77; Homework 3, 73; Exam 1, 70; Exam 2, 72; Participation, 85.

Part 4: After a couple years of teaching this class, you now have grades for 4 classes split into 2 years. You want to compute the average grade of each class, as well as the average grade across all classes in each year. Produce another section in the spreadsheet for Year 2, and calculate the average grade for each year.

Data:

Year 2, Semester 1:

- Julio: Homework 1, 81; Homework 2, 85; Homework 3, 83; Exam 1, 78; Exam 2, 82; Participation, 88.
- Amir: Homework 1, 87; Homework 2, 89; Homework 3, 91; Exam 1, 90; Exam 2, 92; Participation, 94.

Year 2, Semester 2:

- Chloe: Homework 1, 84; Homework 2, 88; Homework 3, 86; Exam 1, 89; Exam 2, 91; Participation, 93.
- Benji: Homework 1, 77; Homework 2, 79; Homework 3, 75; Exam 1, 72; Exam 2, 74; Participation, 85.
- Amelia: Homework 1, 93; Homework 2, 95; Homework 3, 94; Exam 1, 97; Exam 2, 98; Participation, 100.

B.3 Task 3

Part 1: You are planning to cook dinner for a party, and you want to create a spreadsheet with all of the recipes you’ll need. Start with one recipe. Your spreadsheet should include the following:

- Recipe name
- Number of servings
- A table with ingredients and amount of each ingredient required
- A list of steps

Data:

- Recipe name: Cookies

- Number of servings: 2
- Ingredients: 3 cups flour, 2 eggs, 1 cup butter, 1 cup sugar
- Steps: 1. Mix ingredients well. 2. Roll cookies onto a baking sheet. 3. Bake at 375F for 10 minutes. 4. Let cool for 5 minutes.

Part 2: You realized that you need to make more servings than your recipe provides. To solve this, you want to add a scale factor that you can multiply the servings and number of ingredients by (e.g. double the ingredients to make double the servings). Add the scale factor to your spreadsheet, along with a cell computing the new number of servings. Finally, add a column to the table of ingredients showing the new amount of each ingredient after applying the scale factor.

Data:

- Scale factor: 2

Part 3: Now, you want to add more recipes to your recipe book. Add the following two recipes. They should follow the same pattern as the previous one: each recipe should have a recipe name, original number of servings, scale factor, new number of servings, ingredients table (with column applying the scale factor), and list of steps.

Data:

Recipe 1:

- Recipe name: Braised Celery
- Number of servings: 4
- Scale factor: 3
- Ingredients: 1 bunch celery, 2 tablespoons butter, 0.5 cup chicken broth
- Steps: 1. Cut celery into 1-inch pieces. 2. Cook celery in butter over medium heat. 3. Add broth and simmer for 7 minutes.

Recipe 2:

- Recipe name: Ants on a Log
- Number of servings: 10
- Scale factor: 2
- Ingredients: 1 bunch celery, 0.5 cup peanut butter, 0.25 cup raisins
- Steps: 1. Cut celery stalks in half. Spread peanut butter on top. 2. Arrange raisins on top of peanut butter.

Part 4: Finally, you want to create a shopping list of all the ingredients. Use a formula to get a list of the unique ingredient names across all recipes. Ideally, your formula should update automatically when new recipes or ingredients are added.